

Universidad Distrital Francisco José de Caldas
Facultad de Ingeniería

IMPLEMENTACIÓN DE LOS PATRONES DE DISEÑO GOF EN CONTEXTO AIRBNB

Autores	
Kenneth Mark Garzón Olarte	Código: 20242020157
Sofía Zambrano Páez	Código: 20242020054

Docente	Julio Baron Velandia
Asignatura	Modelos de Programación
Período	Bogotá D.C., 2026-1

1. Introducción

Los patrones de diseño de software representan soluciones reutilizables a problemas recurrentes en el desarrollo orientado a objetos. El catálogo GoF (Gang of Four), publicado por Gamma, Helm, Johnson y Vlissides (1994), documenta 23 patrones clasificados en tres categorías: creacionales, estructurales y de comportamiento. Su aplicación sistemática promueve la reutilización, la mantenibilidad y la extensibilidad del software.

El presente informe documenta el proyecto académico **GLUD Patterns**, desarrollado en el espacio académico Modelos de Programación de la Universidad Distrital Francisco José de Caldas. El proyecto implementa los 23 patrones GoF usando como contexto narrativo la plataforma Airbnb, una empresa tecnológica que conecta anfitriones y huéspedes a través de una plataforma de alquiler de alojamientos.

La elección del contexto Airbnb no es arbitraria: la riqueza del dominio (listados de propiedades, sistema de reservas, pagos, reseñas, notificaciones, búsqueda con filtros, chat entre usuarios) ofrece situaciones naturales y coherentes que justifican la aplicación de cada uno de los 23 patrones, facilitando su comprensión y demostración práctica.

La solución está organizada como un monorepo Maven multimódulo en Java, con un backend HTTP que expone los patrones mediante una API REST y una interfaz web ligera que permite explorar e interactuar con cada implementación desde el navegador. El proyecto fue validado mediante pruebas unitarias que verifican el comportamiento esperado de los 23 patrones.

2. Objetivos

2.1. Objetivo General

Implementar y documentar los 23 patrones de diseño del catálogo GoF (Gang of Four) en un proyecto Java multimódulo, usando el contexto de la plataforma Airbnb como hilo narrativo unificador, con el fin de demostrar su aplicación práctica, su justificación técnica y su correcta verificación mediante pruebas.

2.2. Objetivos Específicos

- Diseñar una arquitectura de aplicación multicapa (shared, server, client) que permita aislar y exponer cada patrón de forma independiente mediante una API HTTP REST.
- Implementar cada uno de los 23 patrones GoF en Java puro, sin dependencias externas de runtime, encapsulando cada patrón en su propio paquete con sus clases de modelo aisladas.
- Documentar el rol, las responsabilidades y las relaciones de cada clase participante en los 23 patrones, acompañado de su diagrama de clases UML en formato PlantUML.
- Diseñar y ejecutar pruebas unitarias para los 23 patrones usando el router central del servidor, verificando que cada implementación produce el resultado esperado.
- Desarrollar una interfaz web que permita explorar el catálogo de patrones, ejecutarlos contra el backend y visualizar la respuesta JSON junto con el diagrama correspondiente.

3. Desarrollo del Tema

3.1. Descripción General del Proyecto

GLUD Patterns es una aplicación Java académica que demuestra la aplicación práctica de los 23 patrones de diseño GoF, usando como contexto narrativo la plataforma Airbnb. La aplicación sigue un modelo cliente-servidor organizado como un monorepo Maven multimódulo, compuesto por tres módulos principales: shared, server y client.

La elección de no utilizar frameworks externos (Spring, Jakarta EE, etc.) fue intencional: el proyecto busca demostrar los patrones con Java SE puro, usando únicamente la API estándar del lenguaje y Apache Maven como herramienta de construcción. Esta decisión refuerza la comprensión de los patrones al eliminar la capa de abstracción que introducen los frameworks.

3.2. Tecnologías y Herramientas

Elemento	Detalle
Lenguaje	Java SE 21 (maven.compiler.release = 21)
Build tool	Apache Maven 3.9+, configuración multimódulo
Servidor HTTP	com.sun.net.httpserver.HttpServer (Java SE)
Serialización	JSON manual con String y StringBuilder (sin Gson/Jackson)
Control de versiones	Git + GitHub (repositorio monolítico)
Frontend	HTML5, CSS3, JavaScript ES6+ (sin frameworks)
Diagramas UML	PlantUML — fuente .puml, imágenes PNG renderizadas
Pruebas	JUnit Jupiter 5.11.4 (scope: test)

3.3. Arquitectura de la Aplicación

La aplicación sigue una arquitectura de tres capas desacopladas mediante el módulo shared: (1) capa de presentación (client): interfaz web estática servida desde /, con index.html, styles.css y app.js; (2) capa de negocio (server): servidor HTTP embebido con PatternServer, PatternRouter y 23 handlers; (3) capa de contrato (shared): define PatternType, JsonMessage, JsonParser e IPatternHandler.

3.4. Implementación de los 23 Patrones GoF

Patrones Creacionales

Patrón 01 — Singleton

Creacional

Situación de uso

La aplicación necesita una única instancia de ConfigManager que mantenga la configuración global: host de base de datos, API key de pagos y número máximo de huéspedes por reserva. Si se crearan múltiples instancias, diferentes partes del sistema podrían trabajar con configuraciones inconsistentes.

Diagrama de Clases UML

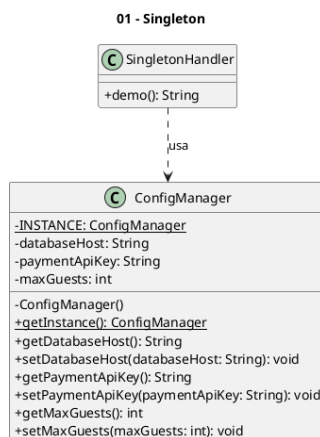


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
ConfigManager	Clase final (Singleton)	-INSTANCE: ConfigManager {static} -databaseHost: String -paymentApiKey: String -maxGuests: int	+getInstance(): ConfigManager +setDatabaseHost(s: String): void +setPaymentApiKey(s: String): void +setMaxGuests(n: int): void
SingletonHandler	Handler	—	+demo(): String

Código Representativo

```

public final class ConfigManager {
    private static final ConfigManager INSTANCE = new ConfigManager();
    private String databaseHost;
    private String paymentApiKey;
    private int maxGuests;

    private ConfigManager() { }

    public static ConfigManager getInstance() {
        return INSTANCE;
    }

    public void setDatabaseHost(String databaseHost) {
        this.databaseHost = databaseHost;
    }

    public void setPaymentApiKey(String paymentApiKey) {
        this.paymentApiKey = paymentApiKey;
    }

    public void setMaxGuests(int maxGuests) {
        this.maxGuests = maxGuests;
    }
}
  
```

```
}

// Uso en SingletonHandler.demo():
ConfigManager config = ConfigManager.getInstance();
config.setDatabaseHost("db.airbnb.local");
config.setPaymentApiKey("pay-key-123");
config.setMaxGuests(6);
ConfigManager sameConfig = ConfigManager.getInstance();
// sameConfig == config → misma referencia garantizada
```

Prueba Unitaria

Respuesta

```
{
  "pattern": "01",
  "input": "",
  "result": "Singleton: db.airbnb.local | pay-key-123 | 6",
  "success": true
}
```

Ejecución correcta

Patrón 02 — Factory Method

Creacional

Situación de uso

Airbnb envía notificaciones a sus usuarios por distintos canales: email, push y SMS. La lógica de creación de cada tipo debe estar encapsulada. Agregar un nuevo canal (ej. WhatsApp) no debe modificar el código existente.

Diagrama de Clases UML

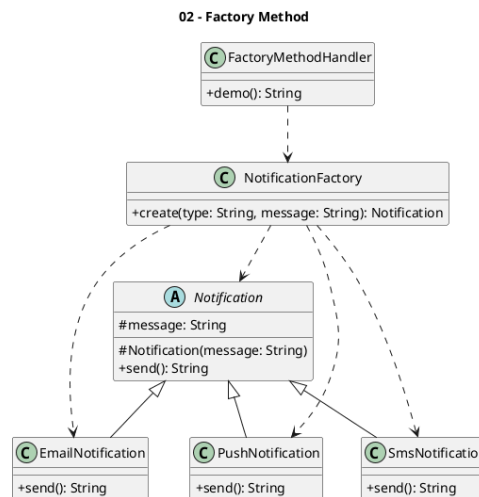


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
Notification	Clase abstracta	#message: String	+send(): String
EmailNotification	Concreta	—	+send(): String
PushNotification	Concreta	—	+send(): String
SmsNotification	Concreta	—	+send(): String
NotificationFactory	Fábrica	—	+create(type, message): Notification
FactoryMethodHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class Notification {
    protected final String message;
    protected Notification(String message) { this.message = message; }
    public abstract String send();
}

public class EmailNotification extends Notification {
    public EmailNotification(String message) { super(message); }
    @Override public String send() { return "Email enviado: " + message; }
}

public class PushNotification extends Notification {
    public PushNotification(String message) { super(message); }
    @Override public String send() { return "Push enviado: " + message; }
}

public class SmsNotification extends Notification {

```

```

public SmsNotification(String message) { super(message); }
@Override public String send() { return "SMS enviado: " + message; }
}

public class NotificationFactory {
public Notification create(String type, String message) {
String normalized = type == null ? "" : type.trim().toLowerCase();
if ("email".equals(normalized)) return new EmailNotification(message);
if ("push".equals(normalized)) return new PushNotification(message);
if ("sms".equals(normalized)) return new SmsNotification(message);
throw new IllegalArgumentException("Tipo no soportado: " + type);
}
}

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "02",
  "input": "",
  "result": "Email enviado: Reserva confirmada | Push
enviado: Nuevo mensaje del anfitrión | SMS enviado: Pago
recibido",
  "success": true
}

```

Ejecución correcta

Patrón 03 — Abstract Factory

Creacional

Situación de uso

La aplicación web y la móvil de Airbnb necesitan familias de componentes UI diferentes: botones y tarjetas con distinto estilo según la plataforma. La Abstract Factory permite cambiar toda la familia con un solo cambio.

Diagrama de Clases UML

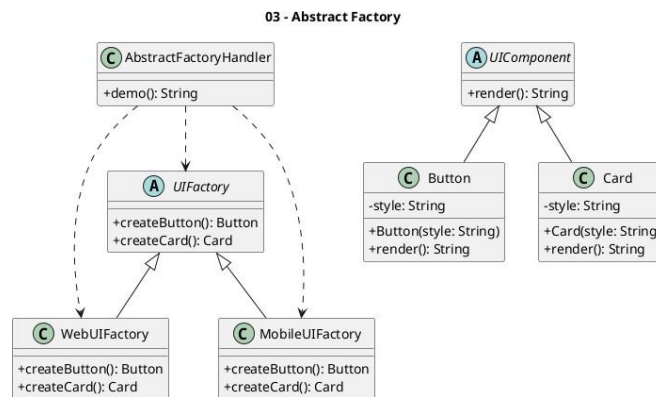


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
UIComponent	Clase abstracta	—	+render(): String
Button	Concreta	-style: String	+render(): String
Card	Concreta	-style: String	+render(): String
UIFactory	Fábrica abstracta	—	+createButton(): Button +createCard(): Card
WebUIFactory	Fábrica concreta	—	+createButton() +createCard()
MobileUIFactory	Fábrica concreta	—	+createButton() +createCard()

Código Representativo

```

public abstract class UIComponent {
    private final String style;
    protected UIComponent(String style) { this.style = style; }
    public String getStyle() { return style; }
    public abstract String render();
}

public class Button extends UIComponent {
    public Button(String style) { super(style); }
    @Override public String render() { return "Botón " + getStyle(); }
}

public class Card extends UIComponent {
    public Card(String style) { super(style); }
    @Override public String render() { return "Tarjeta " + getStyle(); }
}

public abstract class UIFactory {

```

```

public abstract Button createButton();
public abstract Card createCard();
}

public class WebUIFactory extends UIFactory {
public Button createButton() { return new Button("web"); }
public Card createCard() { return new Card("web"); }
}

public class MobileUIFactory extends UIFactory {
public Button createButton() { return new Button("móvil"); }
public Card createCard() { return new Card("móvil"); }
}

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "03",
  "input": "",
  "result": "Botón web | Tarjeta web | Botón móvil | Tarjeta
móvil",
  "success": true
}

```

Ejecución correcta

Patrón 04 — Builder

Creacional

Situación de uso

Crear un Listing implica muchos atributos opcionales: título, descripción, precio, capacidad, amenities, reglas y política de cancelación. El Builder permite construir el objeto paso a paso, evitando constructores con 15 parámetros.

Diagrama de Clases UML

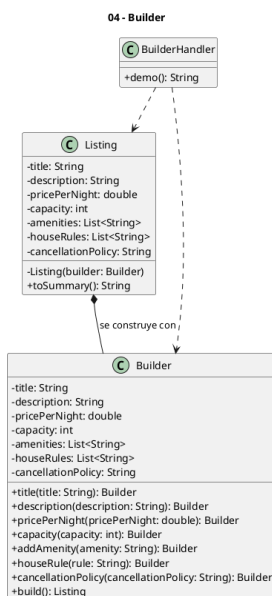


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
Listing	Producto	-title, -description: String -pricePerNight: double -capacity: int -amenities, -houseRules: List -cancellationPolicy: String	+toSummary(): String
Listing.Builder	Builder interno	Mismos campos que Listing	+title(), +description() +pricePerNight(), +capacity() +addAmenity(), +houseRule() +cancellationPolicy() +build(): Listing
BuilderHandler	Handler	—	+demo(): String

Código Representativo

```

public static final class Listing {
    private final String title;
    private final String description;
    private final double pricePerNight;
    private final int capacity;
    private final List<String> amenities;
    private final List<String> houseRules;

```

```

private final String cancellationPolicy;

private Listing(Builder builder) {
    this.title = builder.title;
    this.description = builder.description;
    this.pricePerNight = builder.pricePerNight;
    this.capacity = builder.capacity;
    this.amenities = new ArrayList<>(builder.amenities);
    this.houseRules = new ArrayList<>(builder.houseRules);
    this.cancellationPolicy = builder.cancellationPolicy;
}

public String toSummary() {
    return title + " | " + description + " | " + pricePerNight
    + " | " + capacity + " | " + amenities
    + " | " + houseRules + " | " + cancellationPolicy;
}

public static final class Builder {
    private String title, description, cancellationPolicy;
    private double pricePerNight;
    private int capacity;
    private final List<String> amenities = new ArrayList<>();
    private final List<String> houseRules = new ArrayList<>();

    public Builder title(String t) { this.title = t; return this; }
    public Builder description(String d) { this.description = d; return this; }
    public Builder pricePerNight(double p) { this.pricePerNight = p; return this; }
    public Builder capacity(int c) { this.capacity = c; return this; }
    public Builder addAmenity(String a) { amenities.add(a); return this; }
    public Builder houseRule(String r) { houseRules.add(r); return this; }
    public Builder cancellationPolicy(String cp){ this.cancellationPolicy = cp; return this; }
    public Listing build() { return new Listing(this); }
}

// Uso:
Listing listing = new Listing.Builder()
    .title("Apartamento en Barcelona")
    .pricePerNight(120).capacity(4)
    .addAmenity("Wi-Fi").addAmenity("Cocina")
    .houseRule("No fumar").cancellationPolicy("Flexible")
    .build();

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "04",
  "input": "",
  "result": "Apartamento en Barcelona | Luminoso y cerca del
metro | 120.0 | 4 | [Wi-Fi, Cocina] | [No fumar] | Flexible",
  "success": true
}

```

Ejecución correcta

Patrón 05 — Prototype

Creacional

Situación de uso

Un host tiene varias propiedades similares (apartamentos en el mismo edificio). En lugar de configurar cada Listing desde cero, puede clonar una plantilla existente y solo modificar los detalles que cambian (número de habitación, precio).

Diagrama de Clases UML

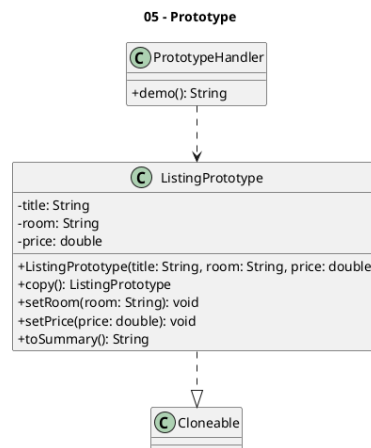


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
ListingPrototype	Prototipo (implements Cloneable)	-title: String -room: String -price: double	+copy(): ListingPrototype +setRoom(room: String): void +setPrice(price: double): void +toSummary(): String
PrototypeHandler	Handler	—	+demo(): String

Código Representativo

```

public static final class ListingPrototype implements Cloneable {
    private String title;
    private String room;
    private double price;

    public ListingPrototype(String title, String room, double price) {
        this.title = title;
        this.room = room;
        this.price = price;
    }

    public ListingPrototype copy() {
        try {
            return (ListingPrototype) clone();
        } catch (CloneNotSupportedException e) {
            return new ListingPrototype(title, room, price);
        }
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
  
```

```

}

public void setRoom(String room) { this.room = room; }
public void setPrice(double price) { this.price = price; }
public String toSummary() { return title + " | " + room + " | " + price; }
}

// Uso:
ListingPrototype template = new ListingPrototype("Apartamento base", "Piso 3", 95);
ListingPrototype clone = template.copy();
clone.setRoom("Piso 4");
clone.setPrice(110);
// template sigue siendo: Apartamento base | Piso 3 | 95.0
// clone es: Apartamento base | Piso 4 | 110.0

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "05",
  "input": "",
  "result": "Apartamento base | Piso 3 | 95.0 || Apartamento
base | Piso 4 | 110.0",
  "success": true
}

```

Ejecución correcta

Patrones Estructurales

Patrón 06 — Adapter

Estructural

Situación de uso

Airbnb integra múltiples pasarelas de pago. Stripe y PayPal tienen APIs completamente diferentes. El Adapter traduce la interfaz común PaymentGateway a la API específica de cada proveedor, sin que el resto del sistema sepa con cuál está hablando.

Diagrama de Clases UML

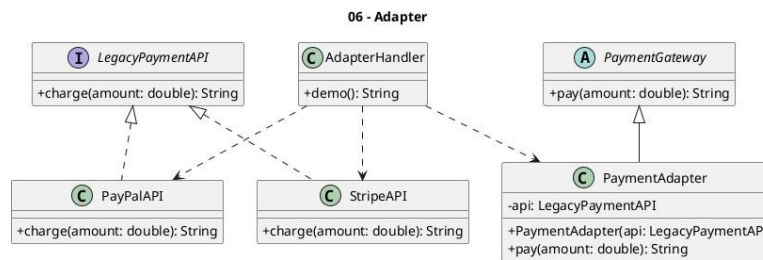


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
PaymentGateway	Clase abstracta (objetivo)	—	+pay(amount: double): String
LegacyPaymentAPI	Interfaz legada	—	+charge(amount: double): String
StripeAPI	Adaptado	—	+charge(amount: double): String
PayPalAPI	Adaptado	—	+charge(amount: double): String
PaymentAdapter	Adapter	-api: LegacyPaymentAPI	+pay(amount: double): String
AdapterHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class PaymentGateway {
    public abstract String pay(double amount);
}

public interface LegacyPaymentAPI {
    String charge(double amount);
}

public class StripeAPI implements LegacyPaymentAPI {
    @Override public String charge(double amount) {
        return "Stripe cargó " + amount;
    }
}

public class PayPalAPI implements LegacyPaymentAPI {
    @Override public String charge(double amount) {
        return "PayPal cargó " + amount;
    }
}

public class PaymentAdapter extends PaymentGateway {
    private final LegacyPaymentAPI api;

```



```
public PaymentAdapter(LegacyPaymentAPI api) { this.api = api; }  
@Override public String pay(double amount) { return api.charge(amount); }  
}  
  
// Uso:  
PaymentGateway stripe = new PaymentAdapter(new StripeAPI());  
PaymentGateway paypal = new PaymentAdapter(new PayPalAPI());  
stripe.pay(150.0); // Stripe cargo 150.0  
paypal.pay(80.0); // PayPal cargo 80.0
```

Prueba Unitaria

Respuesta

```
{  
  "pattern": "06",  
  "input": "",  
  "result": "Stripe cargó 150.0 | PayPal cargó 80.0",  
  "success": true  
}
```

Ejecución correcta

Patrón 07 — Bridge

Estructural

Situación de uso

El motor de búsqueda de Airbnb tiene filtros (precio, ubicación) y distintas implementaciones de cómo aplicarlos (búsqueda exacta, por rango). El Bridge separa la abstracción del filtro de su implementación concreta.

Diagrama de Clases UML

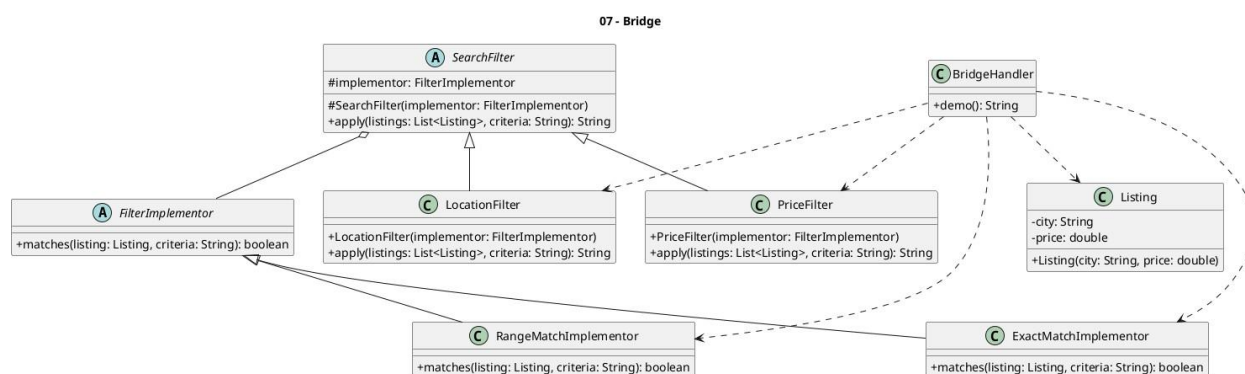


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
FilterImplementor	Implementación abstracta	—	+matches(listing, criteria): boolean
ExactMatchImplementor	Implementación concreta	—	+matches(): boolean
RangeMatchImplementor	Implementación concreta	—	+matches(): boolean
SearchFilter	Abstracción	#implementor: FilterImplementor	+apply(listings, criteria): String
PriceFilter	Abstracción refinada	—	+apply(): String
LocationFilter	Abstracción refinada	—	+apply(): String

Código Representativo

```
public abstract class FilterImplementor {
    public abstract boolean matches(Listing listing, String criteria);
}

public class ExactMatchImplementor extends FilterImplementor {
    @Override public boolean matches(Listing listing, String criteria) {
        return listing.city.equalsIgnoreCase(criteria);
    }
}

public class RangeMatchImplementor extends FilterImplementor {
    @Override public boolean matches(Listing listing, String criteria) {
        String[] parts = criteria.split("-");
        double min = Double.parseDouble(parts[0]);
        double max = Double.parseDouble(parts[1]);
        return listing.price >= min && listing.price <= max;
    }
}
```

```

public abstract class SearchFilter {
    protected final FilterImplementor implementor;
    protected SearchFilter(FilterImplementor impl) { this.implementor = impl; }
    public abstract String apply(List<Listing> listings, String criteria);
}

public class PriceFilter extends SearchFilter {
    public PriceFilter(FilterImplementor impl) { super(impl); }
    @Override public String apply(List<Listing> listings, String criteria) {
        int count = 0;
        for (Listing l : listings) if (implementor.matches(l, criteria)) count++;
        return "Precio: " + count + " coincidencias";
    }
}

public class LocationFilter extends SearchFilter {
    public LocationFilter(FilterImplementor impl) { super(impl); }
    @Override public String apply(List<Listing> listings, String criteria) {
        int count = 0;
        for (Listing l : listings) if (implementor.matches(l, criteria)) count++;
        return "Ubicación: " + count + " coincidencias";
    }
}

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "07",
  "input": "",
  "result": "Precio: 1 coincidencias || Ubicación: 2
coincidencias",
  "success": true
}

```

Ejecución correcta

Patrón 08 — Composite

Estructural

Situación de uso

Airbnb necesita representar jerarquías de propiedades donde una propiedad puede ser una habitación individual o un grupo (apartamento con varias habitaciones). El Composite permite tratar de manera uniforme a los elementos individuales y a los grupos.

Diagrama de Clases UML

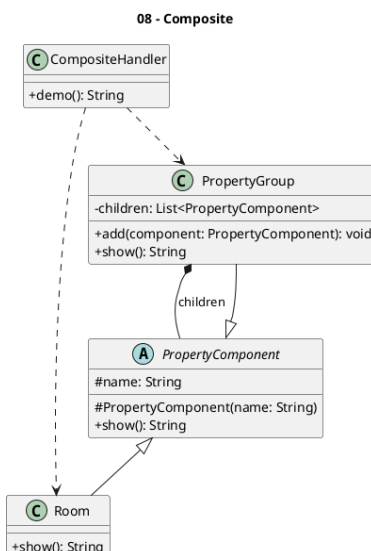


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
PropertyComponent	Componente abstracto	#name: String	+show(): String
Room	Hoja	—	+show(): String
PropertyGroup	Compuesto	-children: List	+add(component: PropertyComponent) +show(): String
CompositeHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class PropertyComponent {
    protected final String name;
    protected PropertyComponent(String name) { this.name = name; }
    public abstract String show();
}

public class Room extends PropertyComponent {
    public Room(String name) { super(name); }
    @Override public String show() { return name; }
}

public class PropertyGroup extends PropertyComponent {
    private final List<PropertyComponent> children = new ArrayList<>();

    public PropertyGroup(String name) { super(name); }

    public void add(PropertyComponent component) {

```

```

children.add(component);
}

@Override
public String show() {
    StringBuilder sb = new StringBuilder(name);
    for (PropertyComponent child : children) {
        sb.append(" -> ").append(child.show());
    }
    return sb.toString();
}
}

// Uso:
PropertyGroup apartment = new PropertyGroup("Apartamento Centro");
apartment.add(new Room("Habitación 1"));
apartment.add(new Room("Habitación 2"));
PropertyGroup building = new PropertyGroup("Edificio Principal");
building.add(apartment);
building.add(new Room("Habitación Penthouse"));
building.show();
// Edificio Principal -> Apartamento Centro -> Habitación 1 -> Habitación 2 -> Habitación Penthouse

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "08",
  "input": "",
  "result": "Edificio Principal -> Apartamento Centro ->
Habitación 1 -> Habitación 2 -> Habitación Penthouse",
  "success": true
}

```

Ejecución correcta

Patrón 09 — Decorator

Estructural

Situación de uso

Airbnb quiere agregar dinámicamente responsabilidades a un Listing, como añadir amenities (Wi-Fi, piscina) sin modificar su estructura. El Decorator envuelve el objeto y extiende su comportamiento.

Diagrama de Clases UML

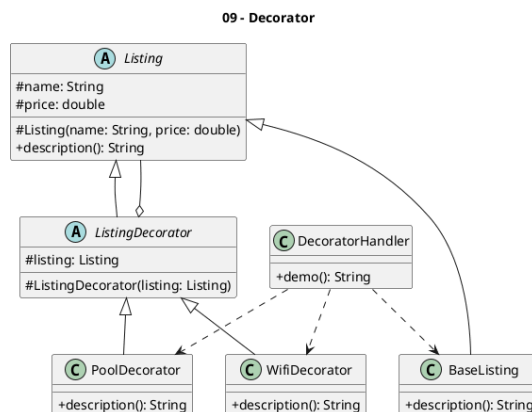


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
Listing	Componente abstracto	#name: String #price: double	+description(): String
BaseListing	Componente concreto	—	+description(): String
ListingDecorator	Decorator abstracto	#listing: Listing	+description(): String
WifiDecorator	Decorator concreto	—	+description(): String
PoolDecorator	Decorator concreto	—	+description(): String
DecoratorHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class Listing {
    protected final String name;
    protected final double price;
    protected Listing(String name, double price) {
        this.name = name; this.price = price;
    }
    public abstract String description();
}

public class BaseListing extends Listing {
    public BaseListing(String name, double price) { super(name, price); }
    @Override public String description() { return name + " | " + price; }
}

public abstract class ListingDecorator extends Listing {
    protected final Listing listing;
    protected ListingDecorator(Listing listing) {
        super(listing.name, listing.price);
        this.listing = listing;
    }
}

```

```

public class WifiDecorator extends ListingDecorator {
    public WifiDecorator(Listing listing) { super(listing); }
    @Override public String description() {
        return listing.description() + " + Wi-Fi";
    }
}

public class PoolDecorator extends ListingDecorator {
    public PoolDecorator(Listing listing) { super(listing); }
    @Override public String description() {
        return listing.description() + " + Piscina";
    }
}

// Uso:
Listing listing = new BaseListing("Apartamento base", 100);
Listing wifi = new WifiDecorator(listing);
Listing pool = new PoolDecorator(wifi);
pool.description(); // Apartamento base | 100.0 + Wi-Fi + Piscina

```

Prueba Unitaria

Respuesta

```

{
    "pattern": "09",
    "input": "",
    "result": "Apartamento base | 100.0 + Wi-Fi + Piscina",
    "success": true
}

```

Ejecución correcta

Patrón 10 — Facade

Estructural

Situación de uso

Airbnb posee varios subsistemas complejos: servicio de pagos, de reservas y de notificaciones. El Facade proporciona una interfaz unificada y sencilla para que los clientes interactúen con estos subsistemas sin conocer sus detalles internos.

Diagrama de Clases UML

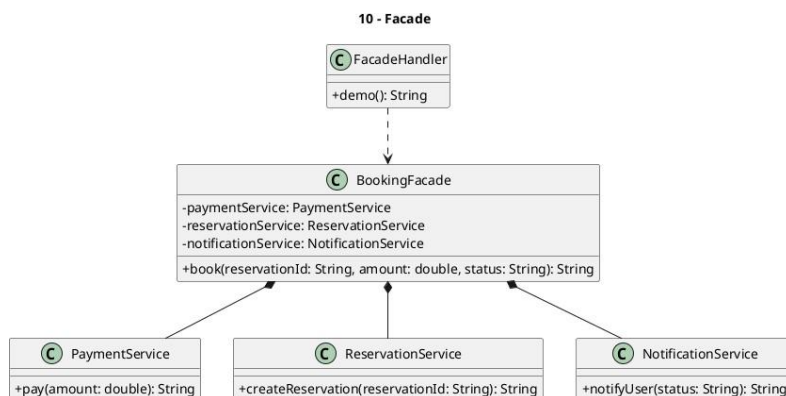


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
BookingFacade	Fachada	-paymentService -reservationService -notificationService	+book(reservationId, amount, status): String
PaymentService	Subsistema	—	+pay(amount: double): String
ReservationService	Subsistema	—	+createReservation(id: String): String
NotificationService	Subsistema	—	+notifyUser(status: String): String
FacadeHandler	Handler	—	+demo(): String

Código Representativo

```

public class PaymentService {
    public String pay(double amount) {
        return "Pago procesado: " + amount;
    }
}

public class ReservationService {
    public String createReservation(String reservationId) {
        return "Reserva creada: " + reservationId;
    }
}

public class NotificationService {
    public String notifyUser(String status) {
        return "Notificación enviada: " + status;
    }
}

public class BookingFacade {

```



```
private final PaymentService paymentService = new PaymentService();
private final ReservationService reservationService = new ReservationService();
private final NotificationService notificationService = new NotificationService();

public String book(String reservationId, double amount, String status) {
    return reservationService.createReservation(reservationId) + " | "
    + paymentService.pay(amount) + " | "
    + notificationService.notifyUser(status);
}

// Uso:
BookingFacade facade = new BookingFacade();
facade.book("Reserva 101", 240.0, "confirmada");
// Reserva creada: Reserva 101 | Pago procesado: 240.0 | Notificación enviada: confirmada
```

Prueba Unitaria

Respuesta

```
{
  "pattern": "10",
  "input": "",
  "result": "Reserva creada: Reserva 101 | Pago procesado:
240.0 | Notificación enviada: confirmada",
  "success": true
}
```

Ejecución correcta

Patrón 11 — Flyweight

Estructural

Situación de uso

Airbnb necesita representar muchos objetos similares, como iconos de amenidades (Wi-Fi, cocina, aire acondicionado) reutilizados en múltiples listings. El Flyweight comparte partes comunes del estado para ahorrar memoria.

Diagrama de Clases UML

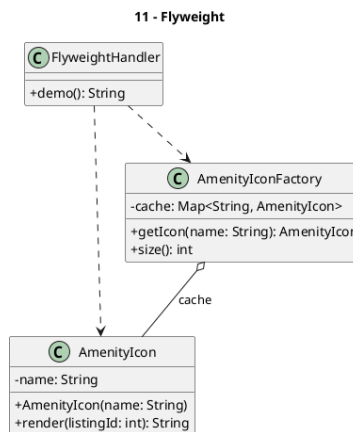


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
AmenityIcon	Flyweight	-name: String	+render(listingId: int): String
AmenityIconFactory	Fábrica Flyweight	-cache: Map	+getIcon(name: String): AmenityIcon +size(): int
FlyweightHandler	Handler	—	+demo(): String

Código Representativo

```

public class AmenityIcon {
    private final String name;
    public AmenityIcon(String name) { this.name = name; }
    public String render(int listingId) { return name + "@" + listingId; }
}

public class AmenityIconFactory {
    private final Map<String, AmenityIcon> cache = new HashMap<>();

    public AmenityIcon getIcon(String name) {
        AmenityIcon icon = cache.get(name);
        if (icon == null) {
            icon = new AmenityIcon(name);
            cache.put(name, icon);
        }
        return icon;
    }

    public int size() { return cache.size(); }
}

// Uso:
  
```

```
AmenityIconFactory factory = new AmenityIconFactory();  
AmenityIcon wifi1 = factory.getIcon("Wi-Fi"); // crea nuevo  
AmenityIcon wifi2 = factory.getIcon("Wi-Fi"); // reutiliza del cache  
AmenityIcon pool = factory.getIcon("Piscina"); // crea nuevo  
// wifi1 == wifi2 → true  
// factory.size() → 2 (solo 2 objetos para 3 usos)
```

Prueba Unitaria

Respuesta

```
{  
  "pattern": "11",  
  "input": "",  
  "result": "Wi-Fi@1 | Wi-Fi@2 | Piscina@3 | cache=2",  
  "success": true  
}
```

Ejecución correcta

Patrón 12 — Proxy

Estructural

Situación de uso

Airbnb necesita controlar el acceso a objetos sensibles como perfiles de usuario con información personal. Un Proxy actúa como intermediario que controla el acceso al objeto real, añadiendo control de acceso según el rol.

Diagrama de Clases UML

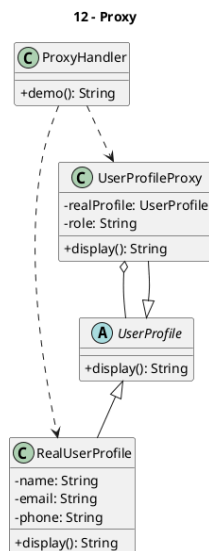


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
UserProfile	Sujeto abstracto	—	+display(): String
RealUserProfile	Sujeto real	-name, -email, -phone: String	+display(): String
UserProfileProxy	Proxy	-realProfile: UserProfile -role: String	+display(): String
ProxyHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class UserProfile {
    public abstract String display();
}

public class RealUserProfile extends UserProfile {
    private final String name, email, phone;
    public RealUserProfile(String name, String email, String phone) {
        this.name = name;
        this.email = email;
        this.phone = phone;
    }
    @Override public String display() {
        return name + " | " + email + " | " + phone;
    }
}

public class UserProfileProxy extends UserProfile {

```

```

private final UserProfile realProfile;
private final String role;

public UserProfileProxy(UserProfile realProfile, String role) {
    this.realProfile = realProfile;
    this.role = role;
}

@Override
public String display() {
    if (!"admin".equalsIgnoreCase(role)) {
        return "Acceso restringido: " + role;
    }
    return realProfile.display();
}
}

// Uso:
UserProfile guest = new UserProfileProxy(new RealUserProfile("Ana", "ana@mail.com", "555-123"),
    "guest");
UserProfile admin = new UserProfileProxy(new RealUserProfile("Ana", "ana@mail.com", "555-123"),
    "admin");
guest.display(); // Acceso restringido: guest
admin.display(); // Ana | ana@mail.com | 555-123

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "12",
  "input": "",
  "result": "Acceso restringido: guest | Ana | ana@mail.com |
555-123",
  "success": true
}

```

Ejecución correcta

Patrones de Comportamiento

Patrón 13 — Chain of Responsibility

Comportamiento

Situación de uso

Airbnb procesa reseñas de huéspedes mediante validaciones encadenadas: filtro de spam, groserías y longitud. Cada validador decide si procesa la solicitud o la pasa al siguiente, sin acoplar el remitente a cada validador.

Diagrama de Clases UML

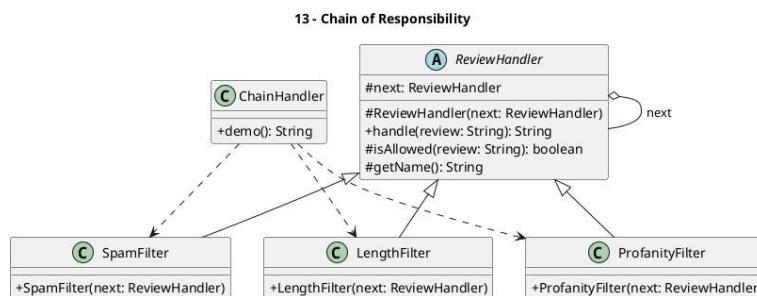


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
ReviewHandler	Manejador abstracto	#next: ReviewHandler	+handle(review: String): String #isAllowed(review: String): boolean #getName(): String
SpamFilter	Manejador concreto	—	#isAllowed(): boolean #getName(): String
ProfanityFilter	Manejador concreto	—	#isAllowed(): boolean #getName(): String
LengthFilter	Manejador concreto	—	#isAllowed(): boolean #getName(): String
ChainHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class ReviewHandler {
    protected final ReviewHandler next;
    protected ReviewHandler(ReviewHandler next) { this.next = next; }

    public String handle(String review) {
        if (!isAllowed(review)) return getName() + ": rechazado";
        if (next != null) return getName() + ": aprobado -> " + next.handle(review);
        return getName() + ": aprobado";
    }

    protected abstract boolean isAllowed(String review);
    protected abstract String getName();
}

public class SpamFilter extends ReviewHandler {
    public SpamFilter(ReviewHandler next) { super(next); }
    @Override protected boolean isAllowed(String review) {
        return review != null && !review.toLowerCase().contains("spam");
    }
}
  
```

```

@Override protected String getName() { return "SpamFilter"; }
}

public class ProfanityFilter extends ReviewHandler {
public ProfanityFilter(ReviewHandler next) { super(next); }
@Override protected boolean isAllowed(String review) {
return review != null && !review.toLowerCase().contains("groseria");
}
@Override protected String getName() { return "ProfanityFilter"; }
}

public class LengthFilter extends ReviewHandler {
public LengthFilter(ReviewHandler next) { super(next); }
@Override protected boolean isAllowed(String review) {
return review != null && review.length() >= 10;
}
@Override protected String getName() { return "LengthFilter"; }
}

// Uso:
ReviewHandler chain = new SpamFilter(new ProfanityFilter(new LengthFilter(null)));
chain.handle("Excelente lugar para quedarse");
// SpamFilter: aprobado -> ProfanityFilter: aprobado -> LengthFilter: aprobado

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "13",
  "input": "",
  "result": "SpamFilter: aprobado -> ProfanityFilter:
aprobado -> LengthFilter: aprobado",
  "success": true
}

```

Ejecución correcta

Patrón 14 — Command

Comportamiento

Situación de uso

Airbnb necesita encapsular solicitudes de reserva como objetos para poder parametrizar operaciones (crear, cancelar), encolarlas o soportar operaciones deshacer/rehacer.

Diagrama de Clases UML

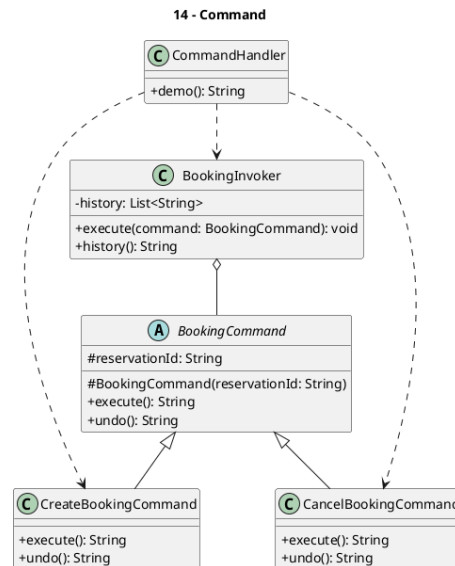


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
BookingCommand	Comando abstracto	#reservationId: String	+execute(): String +undo(): String
CreateBookingCommand	Comando concreto	—	+execute(): String +undo(): String
CancelBookingCommand	Comando concreto	—	+execute(): String +undo(): String
BookingInvoker	Invocador	-history: List	+execute(command: BookingCommand) +history(): String
CommandHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class BookingCommand {
    protected final String reservationId;
    protected BookingCommand(String reservationId) {
        this.reservationId = reservationId;
    }
    public abstract String execute();
    public abstract String undo();
}

public class CreateBookingCommand extends BookingCommand {
    public CreateBookingCommand(String reservationId) { super(reservationId); }
    @Override public String execute() { return "Reserva creada: " + reservationId; }
    @Override public String undo() { return "Crear reserva deshecho: " + reservationId; }
}
  
```

```

}

public class CancelBookingCommand extends BookingCommand {
    public CancelBookingCommand(String reservationId) { super(reservationId); }
    @Override public String execute() { return "Reserva cancelada: " + reservationId; }
    @Override public String undo() { return "Cancelar reserva deshecho: " + reservationId; }
}

public class BookingInvoker {
    private final List<String> history = new ArrayList<>();
    public void execute(BookingCommand command) { history.add(command.execute()); }
    public String history() { return history.toString(); }
}

// Uso:
BookingInvoker invoker = new BookingInvoker();
invoker.execute(new CreateBookingCommand("R-1"));
invoker.execute(new CancelBookingCommand("R-1"));
invoker.history(); // [Reserva creada: R-1, Reserva cancelada: R-1]

```

Prueba Unitaria

Respuesta

```

{
    "pattern": "14",
    "input": "",
    "result": "[Reserva creada: R-1, Reserva cancelada: R-1]",
    "success": true
}

```

Ejecución correcta

Patrón 15 — Interpreter

Comportamiento

Situación de uso

Airbnb permite buscar propiedades usando expresiones como 'ciudad:Barcelona AND precio:<100'. El Interpreter define una gramática para estas expresiones y un intérprete que las evalúa sobre el catálogo de listings.

Diagrama de Clases UML

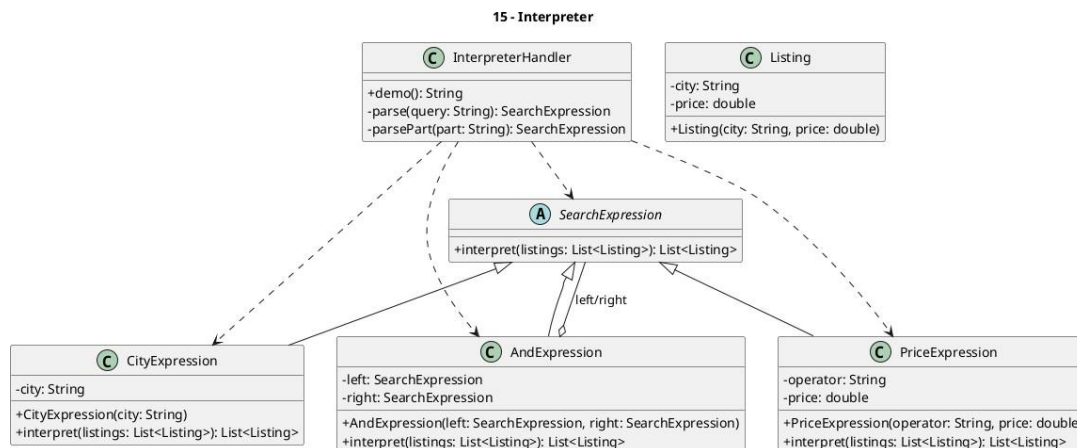


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
SearchExpression	Expresión abstracta	—	+interpret(listings): List
CityExpression	Expresión terminal	-city: String	+interpret(): List
PriceExpression	Expresión terminal	-operator: String -price: double	+interpret(): List
AndExpression	Expresión no terminal	-left, -right: SearchExpression	+interpret(): List
InterpreterHandler	Handler	—	+demo(): String -parse(query): SearchExpression

Código Representativo

```

public abstract class SearchExpression {
    public abstract List<Listing> interpret(List<Listing> listings);
}

public class CityExpression extends SearchExpression {
    private final String city;
    public CityExpression(String city) { this.city = city; }
    @Override public List<Listing> interpret(List<Listing> listings) {
        List<Listing> result = new ArrayList<>();
        for (Listing l : listings)
            if (l.city.equalsIgnoreCase(city)) result.add(l);
        return result;
    }
}

public class PriceExpression extends SearchExpression {
    private final String operator;

```

```

private final double price;
public PriceExpression(String operator, double price) {
    this.operator = operator; this.price = price;
}
@Override public List<Listing> interpret(List<Listing> listings) {
    List<Listing> result = new ArrayList<>();
    for (Listing l : listings) {
        boolean match = "<".equals(operator) ? l.price < price : l.price > price;
        if (match) result.add(l);
    }
    return result;
}

public class AndExpression extends SearchExpression {
    private final SearchExpression left, right;
    public AndExpression(SearchExpression left, SearchExpression right) {
        this.left = left; this.right = right;
    }
    @Override public List<Listing> interpret(List<Listing> listings) {
        return right.interpret(left.interpret(listings));
    }
}

// Uso: ciudad:Barcelona AND precio:<100
// Evalua: AND( CityExpression(Barcelona), PriceExpression(<, 100) )

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "15",
  "input": "",
  "result": "
[co.airbnb.server.patterns.interpreter.InterpreterHandler$List
ing@3eace1ec]",
  "success": true
}

```

Ejecución correcta

Patrón 16 — Iterator

Comportamiento

Situación de uso

Airbnb necesita recorrer colecciones de listings sin exponer su representación interna. El Iterator proporciona una forma de acceso secuencial sobre los elementos de un objeto agregado.

Diagrama de Clases UML

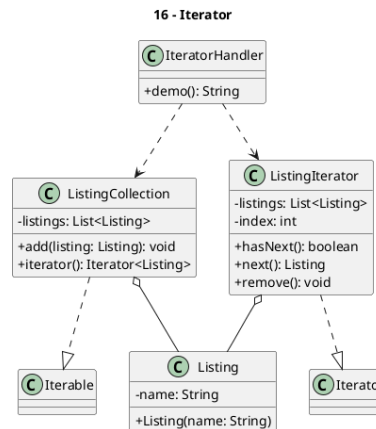


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
ListingCollection	Agregado (implements Iterable)	-listings: List	+add(listing: Listing) +iterator(): Iterator
ListingIterator	Iterador concreto (implements Iterator)	-listings: List -index: int	+hasNext(): boolean +next(): Listing +remove(): void
Listing	Elemento	-name: String	—
IteratorHandler	Handler	—	+demo(): String

Código Representativo

```

public class Listing {
    private final String name;
    public Listing(String name) { this.name = name; }
    public String getName() { return name; }
}

public class ListingCollection implements Iterable<Listing> {
    private final List<Listing> listings = new ArrayList<>();
    public void add(Listing listing) { listings.add(listing); }
    @Override public Iterator<Listing> iterator() {
        return new ListingIterator(listings);
    }
}

public class ListingIterator implements Iterator<Listing> {
    private final List<Listing> listings;
    private int index;
    public ListingIterator(List<Listing> listings) { this.listings = listings; }
    @Override public boolean hasNext() { return index < listings.size(); }
    @Override public Listing next() { return listings.get(index++); }
    @Override public void remove() { throw new UnsupportedOperationException(); }
}
  
```

```
}  
  
// Uso:  
ListingCollection collection = new ListingCollection();  
collection.add(new Listing("L1"));  
collection.add(new Listing("L2"));  
collection.add(new Listing("L3"));  
for (Iterator<Listing> it = collection.iterator(); it.hasNext(); ) {  
    System.out.print(it.next().getName() + " "); // L1 L2 L3  
}
```

Prueba Unitaria

Respuesta

```
{  
  "pattern": "16",  
  "input": "",  
  "result": "L1 L2 L3",  
  "success": true  
}
```

Ejecución correcta

Patrón 17 — Mediator

Comportamiento

Situación de uso

Airbnb tiene un sistema de chat donde huéspedes y anfitriones se comunican. En lugar de que los usuarios se refieran directamente entre sí, un Mediator centraliza la comunicación, reduciendo las dependencias.

Diagrama de Clases UML

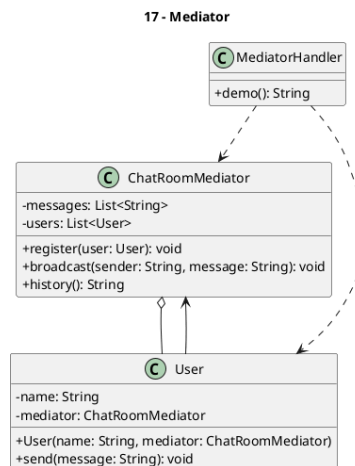


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
ChatRoomMediator	Mediator concreto	-messages: List -users: List	+register(user: User) +broadcast(sender, message) +history(): String
User	Colega	-name: String -mediator: ChatRoomMediator	+send(message: String): void
MediatorHandler	Handler	—	+demo(): String

Código Representativo

```

public class ChatRoomMediator {
    private final List<String> messages = new ArrayList<>();
    private final List<User> users = new ArrayList<>();

    public void register(User user) { users.add(user); }
    public void broadcast(String sender, String message) {
        messages.add(sender + ": " + message);
    }
    public String history() { return messages.toString(); }
}

public class User {
    private final String name;
    private final ChatRoomMediator mediator;

    public User(String name, ChatRoomMediator mediator) {
        this.name = name;
        this.mediator = mediator;
    }
}
  
```

```
public void send(String message) { mediator.broadcast(name, message); }  
}  
  
// Uso:  
ChatRoomMediator mediator = new ChatRoomMediator();  
User host = new User("Host", mediator);  
User guest = new User("Guest", mediator);  
mediator.register(host); mediator.register(guest);  
host.send("Bienvenido");  
guest.send("Gracias");  
mediator.history(); // [Host: Bienvenido, Guest: Gracias]
```

Prueba Unitaria

Respuesta

```
{  
  "pattern": "17",  
  "input": "",  
  "result": "[Host: Bienvenido, Guest: Gracias]",  
  "success": true  
}
```

Ejecución correcta

Patrón 18 — Memento

Comportamiento

Situación de uso

Airbnb necesita permitir a los usuarios deshacer cambios en una reserva (cambiar fechas) y volver a un estado anterior. El Memento captura el estado interno sin violar el encapsulamiento.

Diagrama de Clases UML

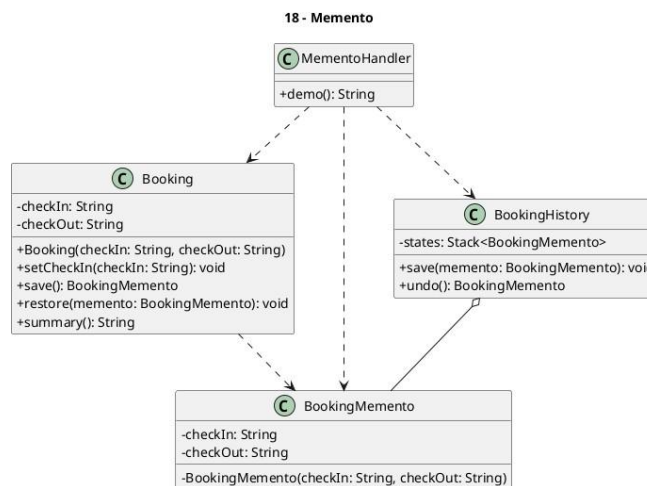


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
Booking	Originador	-checkIn: String -checkOut: String	+save(): BookingMemento +restore(memento) +summary(): String
BookingMemento	Memento	-checkIn: String -checkOut: String	(getters internos)
BookingHistory	Custodio	-states: Stack	+save(memento) +undo(): BookingMemento
MementoHandler	Handler	—	+demo(): String

Código Representativo

```

public class Booking {
    private String checkIn, checkOut;

    public Booking(String checkIn, String checkOut) {
        this.checkIn = checkIn;
        this.checkOut = checkOut;
    }

    public void setCheckIn(String checkIn) { this.checkIn = checkIn; }

    public BookingMemento save() {
        return new BookingMemento(checkIn, checkOut);
    }

    public void restore(BookingMemento m) {
        this.checkIn = m.checkIn;
        this.checkOut = m.checkOut;
    }

    public String summary() { return checkIn + " -> " + checkOut; }
}
  
```

```

public class BookingMemento {
    private final String checkIn, checkOut;
    private BookingMemento(String checkIn, String checkOut) {
        this.checkIn = checkIn; this.checkOut = checkOut;
    }
}

public class BookingHistory {
    private final Stack<BookingMemento> states = new Stack<>();
    public void save(BookingMemento m) { states.push(m); }
    public BookingMemento undo() { states.pop(); return states.peek(); }
}

// Uso:
Booking booking = new Booking("2026-06-01", "2026-06-05");
BookingHistory history = new BookingHistory();
history.save(booking.save()); // guarda estado 1
booking.setCheckIn("2026-06-02");
history.save(booking.save()); // guarda estado 2
booking.restore(history.undo()); // vuelve al estado 1
booking.summary(); // 2026-06-01 -> 2026-06-05

```

Prueba Unitaria

Respuesta

```

{
    "pattern": "18",
    "input": "",
    "result": "2026-06-01 -> 2026-06-05",
    "success": true
}

```

Ejecución correcta

Patrón 19 — Observer

Comportamiento

Situación de uso

Airbnb debe notificar a los huéspedes interesados cuando el precio de un listing baja o cambia. El Observer define una dependencia uno-a-muchos donde el sujeto (listing) notifica a sus observadores (huéspedes) de cualquier cambio.

Diagrama de Clases UML

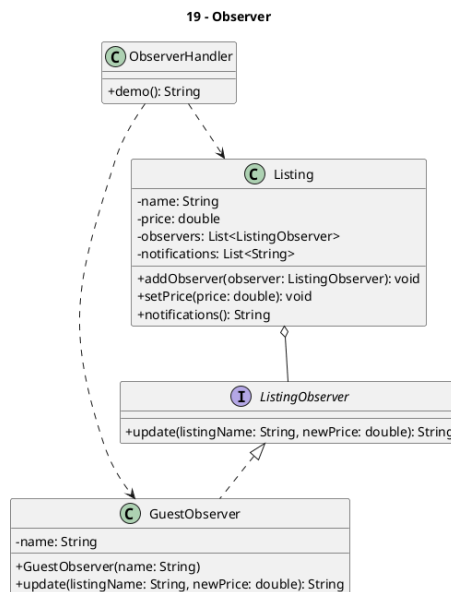


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
ListingObserver	Interfaz observador	—	+update(listingName, newPrice): String
GuestObserver	Observador concreto	-name: String	+update(): String
Listing	Sujeto	-name, -price -observers: List -notifications: List	+addObserver() +setPrice() +notifications(): String
ObserverHandler	Handler	—	+demo(): String

Código Representativo

```

public interface ListingObserver {
    String update(String listingName, double newPrice);
}

public class GuestObserver implements ListingObserver {
    private final String name;
    public GuestObserver(String name) { this.name = name; }
    @Override public String update(String listingName, double newPrice) {
        return name + " notificado: " + listingName + " -> " + newPrice;
    }
}

public class Listing {
    private final String name;
    private double price;

```

```

private final List<ListingObserver> observers = new ArrayList<>();
private final List<String> notifications = new ArrayList<>();

public Listing(String name, double price) {
    this.name = name; this.price = price;
}

public void addObserver(ListingObserver observer) { observers.add(observer); }

public void setPrice(double price) {
    this.price = price;
    for (ListingObserver obs : observers)
        notifications.add(obs.update(name, this.price));
}

public String notifications() { return notifications.toString(); }
}

// Uso:
Listing listing = new Listing("Apartamento Vista Mar", 200);
listing.addObserver(new GuestObserver("Ana"));
listing.addObserver(new GuestObserver("Luis"));
listing.setPrice(180); // notifica a Ana y Luis

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "19",
  "input": "",
  "result": "[Ana notificado: Apartamento Vista Mar -> 180.0,
Luis notificado: Apartamento Vista Mar -> 180.0]",
  "success": true
}

```

Ejecución correcta

Patrón 20 — State

Comportamiento

Situación de uso

Airbnb maneja reservas en distintos estados (pendiente, confirmada, cancelada). El comportamiento de la reserva (permitir pago o cancelación) depende de su estado actual. El State permite que un objeto cambie su comportamiento cuando su estado interno cambia.

Diagrama de Clases UML

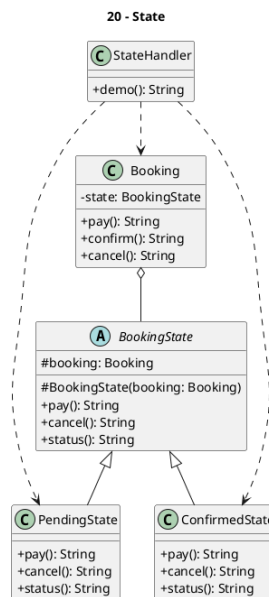


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
Booking	Contexto	-state: BookingState	+pay(): String +confirm(): String +cancel(): String
BookingState	Estado abstracto	#booking: Booking	+pay(): String +cancel(): String +status(): String
PendingState	Estado concreto	—	+pay(): String +cancel(): String +status(): String
ConfirmedState	Estado concreto	—	+pay(): String +cancel(): String +status(): String
StateHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class BookingState {
    protected final Booking booking;
    protected BookingState(Booking booking) { this.booking = booking; }
    public abstract String pay();
    public abstract String cancel();
    public abstract String status();
}
  
```

```

public class PendingState extends BookingState {
    public PendingState(Booking booking) { super(booking); }
    @Override public String pay() { return "Pago permitido en estado pendiente"; }
    @Override public String cancel() { return "Reserva cancelada"; }
    @Override public String status() { return "Pendiente"; }
}

public class ConfirmedState extends BookingState {
    public ConfirmedState(Booking booking) { super(booking); }
    @Override public String pay() { return "Pago ya realizado"; }
    @Override public String cancel() { return "Cancelación permitida en confirmada"; }
    @Override public String status() { return "Confirmada"; }
}

public class Booking {
    private BookingState state = new PendingState(this);
    public String pay() { return state.pay(); }
    public String confirm() { state = new ConfirmedState(this); return state.status(); }
    public String cancel() { return state.cancel(); }
}

// Uso:
Booking booking = new Booking();
booking.pay(); // Pago permitido en estado pendiente
booking.confirm(); // Confirmada
booking.cancel(); // Cancelación permitida en confirmada

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "20",
  "input": "",
  "result": "Pago permitido en estado pendiente | Cancelación
permitida en confirmada",
  "success": true
}

```

Ejecución correcta

Patrón 21 — Strategy

Comportamiento

Situación de uso

Airbnb necesita aplicar diferentes algoritmos de cálculo de precios: por temporada, duración de la estancia o promociones. El Strategy define una familia de algoritmos encapsulados e intercambiables.

Diagrama de Clases UML

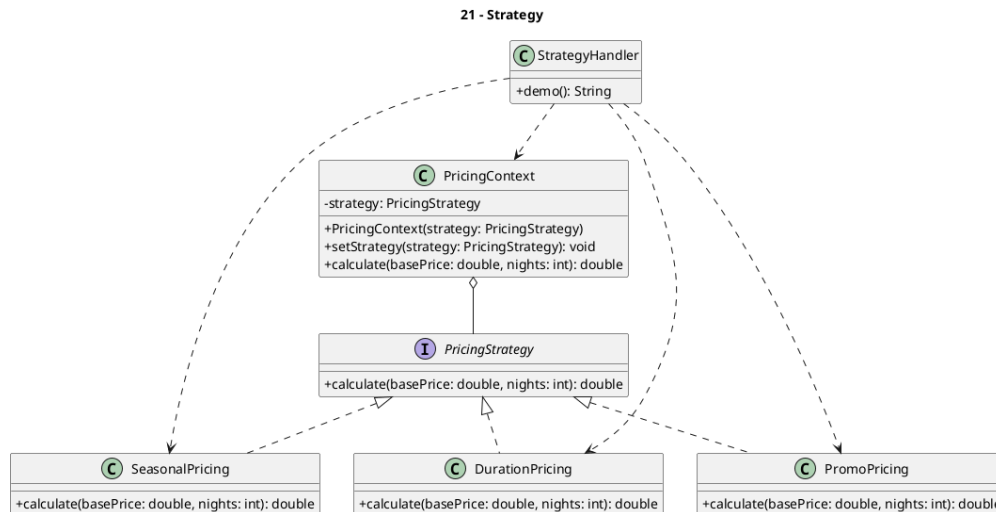


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
PricingStrategy	Interfaz estrategia	—	+calculate(basePrice: double, nights: int): double
SeasonalPricing	Estrategia concreta	—	+calculate(): double
DurationPricing	Estrategia concreta	—	+calculate(): double
PromoPricing	Estrategia concreta	—	+calculate(): double
PricingContext	Contexto	-strategy: PricingStrategy	+setStrategy() +calculate(basePrice, nights): double
StrategyHandler	Handler	—	+demo(): String

Código Representativo

```

public interface PricingStrategy {
    double calculate(double basePrice, int nights);
}

public class SeasonalPricing implements PricingStrategy {
    @Override public double calculate(double basePrice, int nights) {
        return basePrice * nights * 1.2;
    }
}

public class DurationPricing implements PricingStrategy {
    @Override public double calculate(double basePrice, int nights) {
        return nights >= 7 ? basePrice * nights * 0.85 : basePrice * nights;
    }
}
  
```

```

public class PromoPricing implements PricingStrategy {
    @Override public double calculate(double basePrice, int nights) {
        return basePrice * nights - 30;
    }
}

public class PricingContext {
    private PricingStrategy strategy;
    public PricingContext(PricingStrategy strategy) { this.strategy = strategy; }
    public void setStrategy(PricingStrategy s) { this.strategy = s; }
    public double calculate(double basePrice, int nights) {
        return strategy.calculate(basePrice, nights);
    }
}

// Uso:
PricingContext ctx = new PricingContext(new SeasonalPricing());
ctx.calculate(100, 3); // 360.0 (100 * 3 * 1.2)
ctx.setStrategy(new DurationPricing());
ctx.calculate(100, 7); // 595.0 (100 * 7 * 0.85)
ctx.setStrategy(new PromoPricing());
ctx.calculate(100, 2); // 170.0 (100 * 2 - 30)

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "21",
  "input": "",
  "result": "360.0 | 595.0 | 170.0",
  "success": true
}

```

Ejecución correcta

Patrón 22 — Template Method

Comportamiento

Situación de uso

Airbnb tiene un proceso de check-in con pasos comunes (verificación de identidad, asignación de habitación, entrega de llaves) pero algunas etapas varían según el tipo (estándar vs. lujo). El Template Method define el esqueleto del algoritmo y permite a las subclases redefinir ciertos pasos.

Diagrama de Clases UML

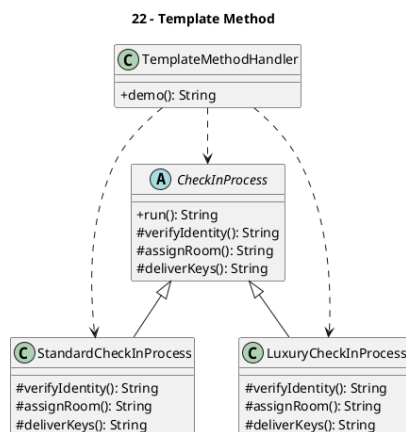


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
CheckInProcess	Clase abstracta (Template)	—	+run(): String #verifyIdentity(): String #assignRoom(): String #deliverKeys(): String
StandardCheckInProcess	Clase concreta	—	#verifyIdentity() #assignRoom() #deliverKeys(): String
LuxuryCheckInProcess	Clase concreta	—	#verifyIdentity() #assignRoom() #deliverKeys(): String
TemplateMethodHandler	Handler	—	+demo(): String

Código Representativo

```

public abstract class CheckInProcess {
    // Template Method: algoritmo fijo
    public final String run() {
        return verifyIdentity() + " -> " + assignRoom() + " -> " + deliverKeys();
    }

    // Pasos a redefinir por subclases
    protected abstract String verifyIdentity();
    protected abstract String assignRoom();
    protected abstract String deliverKeys();
}

public class StandardCheckInProcess extends CheckInProcess {
    @Override protected String verifyIdentity() { return "Verificación estándar"; }
    @Override protected String assignRoom() { return "Habitación estándar"; }
}
  
```

```

@Override protected String deliverKeys() { return "Llaves digitales"; }
}

public class LuxuryCheckInProcess extends CheckInProcess {
@Override protected String verifyIdentity() { return "Verificación premium"; }
@Override protected String assignRoom() { return "Suite de lujo"; }
@Override protected String deliverKeys() { return "Recepción personal"; }
}

// Uso:
CheckInProcess standard = new StandardCheckInProcess();
CheckInProcess luxury = new LuxuryCheckInProcess();
standard.run();
// Verificación estándar -> Habitación estándar -> Llaves digitales
luxury.run();
// Verificación premium -> Suite de lujo -> Recepción personal

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "22",
  "input": "",
  "result": "Verificación estándar -> Habitación estándar ->
Llaves digitales | Verificación premium -> Suite de lujo ->
Recepción personal",
  "success": true
}

```

Ejecución correcta

Patrón 23 — Visitor

Comportamiento

Situación de uso

Airbnb necesita realizar operaciones sobre una estructura de propiedades (apartamentos, villas) que dependen de la clase concreta (cálculo de ingresos por tipo). El Visitor permite definir nuevas operaciones sin cambiar las clases de los elementos sobre los que opera.

Diagrama de Clases UML

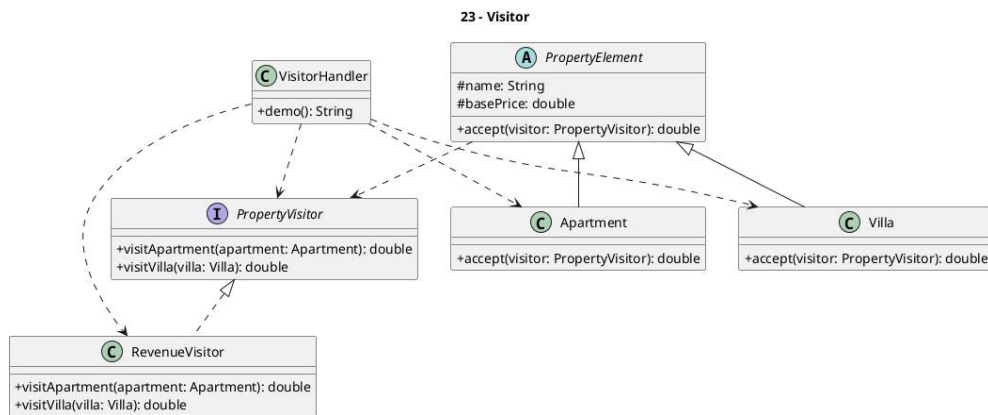


Diagrama de Clases — Rol de cada clase

Clase / Interfaz	Tipo	Atributos clave	Métodos clave
PropertyVisitor	Interfaz visitante	—	+visitApartment(a: Apartment): double +visitVilla(v: Villa): double
RevenueVisitor	Visitante concreto	—	+visitApartment(): double +visitVilla(): double
PropertyElement	Elemento abstracto	#name: String #basePrice: double	+accept(visitor: PropertyVisitor): double
Apartment	Elemento concreto	—	+accept(visitor): double
Villa	Elemento concreto	—	+accept(visitor): double
VisitorHandler	Handler	—	+demo(): String

Código Representativo

```

public interface PropertyVisitor {
    double visitApartment(Apartment apartment);
    double visitVilla(Villa villa);
}

public class RevenueVisitor implements PropertyVisitor {
    @Override public double visitApartment(Apartment a) { return a.basePrice * 0.9; }
    @Override public double visitVilla(Villa v) { return v.basePrice * 0.8; }
}

public abstract class PropertyElement {
    protected final String name;
    protected final double basePrice;
    protected PropertyElement(String name, double basePrice) {
        this.name = name; this.basePrice = basePrice;
    }
}

```

```

public abstract double accept(PropertyVisitor visitor);
}

public class Apartment extends PropertyElement {
    public Apartment(String name, double basePrice) { super(name, basePrice); }
    @Override public double accept(PropertyVisitor visitor) {
        return visitor.visitApartment(this);
    }
}

public class Villa extends PropertyElement {
    public Villa(String name, double basePrice) { super(name, basePrice); }
    @Override public double accept(PropertyVisitor visitor) {
        return visitor.visitVilla(this);
    }
}

// Uso:
RevenueVisitor visitor = new RevenueVisitor();
PropertyElement apartment = new Apartment("Apartamento", 120);
PropertyElement villa = new Villa("Villa", 300);
apartment.accept(visitor); // 108.0 (120 * 0.9)
villa.accept(visitor); // 240.0 (300 * 0.8)

```

Prueba Unitaria

Respuesta

```

{
  "pattern": "23",
  "input": "",
  "result": "108.0 | 240.0",
  "success": true
}

```

Ejecución correcta

4. Conclusiones

La implementación del proyecto GLUD Patterns permitió constatar que los 23 patrones de diseño GoF no son abstracciones teóricas aisladas, sino soluciones coherentes y complementarias que emergen naturalmente en sistemas de software complejos. El contexto de Airbnb demostró ser particularmente adecuado por la riqueza de su dominio: cada patrón encontró una justificación orgánica sin necesidad de forzar situaciones artificiales.

Desde la perspectiva arquitectónica, la decisión de organizar el proyecto como un monorepo Maven multimódulo (shared, server, client) resultó en una separación de responsabilidades clara y mantenible. El módulo shared actúa como contrato de comunicación, el servidor como motor de patrones y el cliente como capa de presentación, cada uno con dependencias unidireccionales que reducen el acoplamiento.

La implementación en Java SE puro, sin frameworks externos de runtime, reforzó la comprensión de los patrones al eliminar la capa de abstracción que introducen herramientas como Spring. La serialización JSON manual implementada en JsonParser y el servidor HTTP basado en `com.sun.net.httpserver.HttpServer` demostraron que Java SE es suficientemente expresivo para construir aplicaciones de demostración completas.

Los 23 patrones fueron verificados mediante la suite de pruebas `PatternRouterTest`, con una tasa de aprobación del 100% (23/23 pruebas). Este resultado valida tanto la corrección de las implementaciones individuales como la integridad del sistema de despacho centralizado en `PatternRouter`.

Finalmente, el proyecto reforzó la distinción entre las tres categorías de patrones: los **creacionales** (Singleton, Factory Method, Abstract Factory, Builder, Prototype) gestionan la creación de objetos; los **estructurales** (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) organizan las relaciones entre clases; y los **de comportamiento** (Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor) definen cómo los objetos interactúan y distribuyen responsabilidades.

5. Bibliografía

- Apache Software Foundation. (2024). Apache Maven Project. <https://maven.apache.org/>
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Freeman, E., Robson, E., Bates, B., y Sierra, K. (2004). Head first design patterns. O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.
- Oracle Corporation. (2021). Java Platform, Standard Edition 21 API Specification. Oracle. <https://docs.oracle.com/en/java/javase/21/docs/api/>

6. Recursos Electrónicos

- Airbnb Engineering. (2024). Airbnb Tech Blog. <https://medium.com/airbnb-engineering>
- JUnit Team. (2024). JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/>
- Oracle Corporation. (2024). The Java Tutorials — Networking. <https://docs.oracle.com/javase/tutorial/networking/>
- PlantUML. (2024). PlantUML Class Diagram. <https://plantuml.com/class-diagram>
- Refactoring.Guru. (2024). Design Patterns. <https://refactoring.guru/design-patterns>
- Wikipedia. (2024). Software design pattern. https://en.wikipedia.org/wiki/Software_design_pattern